

Recherche de documents « *Document Retrieval* »

Séance 2

Prétraitement du texte

I. Introduction :

Lors de la séance précédente, nous avons constitué et exploré un corpus textuel structuré, bilingue et hiérarchisé. Avant de pouvoir analyser, indexer ou vectoriser ces documents, il est indispensable de nettoyer et normaliser les textes (*text cleaning*) afin d'éliminer le bruit, et d'assurer une homogénéité linguistique minimale.

Elle constitue la première phase de prétraitement linguistique et prépare les données pour la tokenisation et la représentation vectorielle. Cette séance introduit les principales opérations de prétraitement linguistique, réalisées sans bibliothèques NLP externes, pour bien comprendre le fonctionnement interne des moteurs de recherche textuelle.

Selon les sources, cette phase peut précéder ou suivre la tokenisation ; dans le cadre de notre projet, nous la plaçons avant, afin de travailler sur un texte brut homogène et propre.

II. Contexte général

On dispose d'une base de textes comportant deux configurations :

a) **Configuration 1 « Corpus de phrases » :**

- Deux fichiers texte : un en français, un en anglais.
- Chaque phrase du fichier est considérée comme un document.
- Le texte doit être d'abord segmenté en phrases, puis nettoyé.

b) **Configuration 2 :**

- Quatre sous-corpus, chacun contenant des documents en français et leurs traductions anglaises.
- Chaque document est une unité entière.
- Le nettoyage est effectué avant la tokenisation en mots.

III. Objectifs de la séance

1. Concevoir un pipeline complet de prétraitement de texte.
2. Écrire des fonctions Python modulaires pour chaque transformation.
3. Comparer l'effet de ces transformations sur la qualité et la cohérence du corpus.

IV. Plan général du prétraitement

Le pipeline complet suit une logique linguistique et technique :

Étape	Objectif	Fonctions principales
1	Chargement et uniformisation	lire_document, convertir_vers_minuscule
2	Suppression des balises HTML/XML	supprimer_balises_html
3	Normalisation Unicode	normaliser_unicode
4	Détection de la langue	detecter_langue_nom_fichier, verifier_langue_contenu
5	Nettoyage structurel (ponctuation, nombres, symboles)	traiter_ponctuation, traiter_nombres,traiter_symboles
6	Nettoyage Web et caractères spéciaux	traiter_web_et_emojis
7	Expansion et correction linguistique	traiter_contractions,developper_abreviations, corriger_fautes_typographiques
8	Normalisation des espaces et de la mise en forme	normaliser_mise_en_forme
9	Suppression des mots vides	supprimer_mots_vides, construire_liste_stopwords
10	Vérification de cohérence post-nettoyage	signaler_incoherences_langue

V. Différentes étapes du nettoyage de texte :

Le prétraitement est une étape fondamentale du « *document retrieval* ». Il s'agit de convertir le texte brut en une forme plus homogène et plus facile à analyser : suppression des caractères parasites, normalisation des mots, découpage en unités lexicales. Les fonctions suivantes permettent d'implémenter manuellement chaque transformation.

1. Chargement et uniformisation du texte brut :

- Écrire une fonction **lire_document(fichier)** qui ouvre un fichier texte et retourne son contenu sous forme de chaîne.
- Écrire une fonction **convertir_vers_minuscule(texte)** qui convertit tout le texte en minuscules pour uniformiser les comparaisons.

2. Suppression des balises HTML/XML

- Écrire une fonction **supprimer_balises_html_xml(texte)** qui retire les balises HTML/XML éventuelles.

3. Normalisation Unicode

Les textes peuvent contenir des caractères visuellement identiques mais encodés différemment selon la source (ex. fichiers provenant du Web, de Word, ou copiés depuis un PDF). Ces différences invisibles peuvent créer des erreurs lors de la comparaison, de la recherche ou de la tokenisation.

La normalisation Unicode garantit que tous les caractères sont représentés de manière cohérente dans la même forme.

"école" (e + accent séparé) → "école" (accent intégré)

"coeur" → "cœur" (recomposition correcte)

- ◆ **NFC** : caractères composés (recommandé pour la plupart des usages)
- ◆ **NFD** : caractères décomposés (utile pour supprimer les accents ensuite)

- a. Écrire une fonction **normaliser_unicode(texte)** qui normalise le texte selon la forme Unicode standard (par défaut NFC). Cette opération uniformise les représentations internes des caractères accentués ou combinés. Utiliser le module standard « `unicodedata` »
- Cela permet d'uniformiser les caractères accentués (é, è, ê) et d'éviter les doublons invisibles.

4. Correction et uniformisation des accents

Les textes issus de sources variées peuvent contenir des erreurs d'encodage (caractères mal interprétés) ou des variations d'accents et de symboles. Cette étape vise à corriger les anomalies liées à l'encodage et à uniformiser les formes accentuées afin d'assurer une cohérence linguistique dans tout le corpus.

- a. Écrire une fonction **corriger_accents(texte)** qui corrige les erreurs d'affichage dues à un mauvais encodage ou décodage du texte (ex. fichiers en ISO-8859-1 lus comme UTF-8). Vous pouvez créer un dictionnaire de remplacements pour les caractères erronés fréquents. Ou tester différentes combinaisons de `.encode()` et `.decode()` pour corriger automatiquement ces erreurs.
- Exemple : "FranÃ§ais" → "Français" ; "dÃ©jÃ " → "déjà"
- b. Écrire une fonction **uniformiser_accents(texte)** qui remplace toutes les variantes d'un même caractère accentué par une forme unique (par exemple, garder uniquement l'accent aigu é pour les comparaisons lexicales). Vous pouvez créer un dictionnaire de remplacements pour les caractères erronés fréquents
- Exemple : "é", "è", "ê", "ë" → "é" ; "à", "â" → "a" ; "ç" → "c"
- a. Écrire une fonction **traiter_accents(texte, options)** qui combine les deux opérations précédentes selon un dictionnaire de paramètres :

```
options = { "corriger_erreurs": True, "uniformiser": True, "langue": "fr" }  
  
texte_nettoye = traiter_accents(texte, options)
```

5. Détection de la langue

Il est possible de vérifier la langue d'un document soit à partir de son nom de fichier (suffixe `_fr` ou `_en`), soit à partir de son contenu textuel (présence de mots caractéristiques de chaque langue).

- Écrire une fonction **detecter_langue_nom_fichier(fichier)** qui déduit la langue du document à partir de son nom (par exemple : `_fr` → français, `_en` → anglais).
- Écrire une fonction **verifier_coherence_langue_contenu(fichier)** qui compare la langue attendue (déduite du nom du fichier) avec celle estimée à partir du contenu textuel, en s'appuyant sur la présence de mots fréquents typiques de chaque langue.
- Écrire une fonction **verifier_langue_contenu(fichier)** qui détermine directement la langue dominante d'un document à partir de son contenu, sans utiliser son nom.
- Écrire une fonction **signaler_incoherences_langue(base)** qui parcourt l'ensemble du corpus et signale les fichiers dont la langue détectée diffère de celle attendue selon le nom du fichier.

6. Nettoyage structurel (ponctuation, nombres, symboles)

◆ Traitement de la ponctuation

- Écrire une fonction **supprimer_ponctuation(texte)** qui supprime toutes les ponctuations du texte (`.`, `,`, `;`, `:`, `!`, `?`, `(`, `)`, `[`, `]`, `"`, `'` ... etc.). On pourra utiliser le module `string.punctuation` ou une expression régulière (`re.sub`).
- Écrire une fonction **remplacer_ponctuation(texte, balise="<PONCT>")** qui remplace chaque signe de ponctuation par une balise donnée (par défaut `<PONCT>`). Exemple : `"Bonjour!"` → `"Bonjour <PONCT>."`
- Écrire une fonction **supprimer_ponctuation_sauf(texte, ponct_a_conserver=[".", "?"])** qui supprime toutes les ponctuations sauf celles spécifiées dans une liste. Exemple : `"Salut! Tu vas bien?"` → `"Salut Tu vas bien?"` si on garde seulement `"?"`.
- Écrire une fonction **espacer_ponctuation(texte)** qui ajoute un espace avant et après chaque signe de ponctuation collé à un mot. Exemple : `"Bonjour,comment ça va?"` → `"Bonjour , comment ça va ?"`. Utile pour améliorer la tokenisation en mots.
- Écrire une fonction **normaliser_ponctuation(texte)** qui uniformise les variantes typographiques :
 - remplacer les guillemets français et anglais (`«`, `»`, `”`, `”`) par `"`;
 - remplacer les tirets longs (`-`, `-`) par un simple `-`;
 - uniformiser les points de suspension (`...` → `"..."`).
- Écrire une fonction **reduire_ponctuation_multiple(texte)** qui remplace les répétitions de ponctuation par une seule occurrence. Exemple : `"!!!"` → `"!"`, `"...."` → `"."`, `"???"` → `"?"`.

- g. Écrire une fonction **remplacer_ponctuation_contextuelle(texte)** qui remplace certaines ponctuations par des balises sémantiques :

- "!" → <EXCLAMATION>
- "?" → <QUESTION>
- "..." → <HESITATION>

Exemple : "Vraiment?!..." → "Vraiment <QUESTION> <EXCLAMATION> <HESITATION>"

- h. Écrire une fonction **traiter_ponctuation(texte, options)** qui combine les fonctions précédentes selon un dictionnaire de paramètres (Cette fonction appellera dynamiquement les autres selon les besoins.) :

```
options = {
    "supprimer": False,
    "remplacer": True,
    "balise": "<PONCT>",
    "espacer": True,
    "normaliser": True,
    "reduire": True,
    "contextuel": False,
    "langue": "fr"
}
texte_nettoye = traiter_ponctuation(texte, options)
```

◆ Traitement des nombres et des symboles :

- a. Écrire une fonction **supprimer_nombres(texte)** qui supprime tous les chiffres et nombres (entiers ou décimaux) du texte.
- Exemple : "En 2025, il aura 3 ans." → "En , il aura ans."
 - On peut utiliser une expression régulière : `re.sub(r'\d+', "", texte)`
- b. Écrire une fonction **remplacer_nombres(texte, balise="<NUM>")** qui remplace chaque nombre par une balise donnée.
- Exemple : "Le prix est de 30 euros" → "Le prix est de <NUM> euros"
- c. Écrire une fonction **nombres_en_lettres(texte, langue="fr")** qui convertit les nombres en leur forme écrite :
- "3" → "trois", "2025" → "deux mille vingt-cinq".
 - Cette fonction peut utiliser la bibliothèque `num2words` (ou une implémentation simplifiée).
 - Exemple : "Il a 2 enfants" → "Il a deux enfants"
- d. Écrire une fonction **supprimer_symboles(texte)** qui supprime les symboles non alphanumériques (ex. @, #, \$, %, *, =, +, <, >...).
- Exemple : "#Hello@World!" → "HelloWorld"

- e. Écrire une fonction **remplacer_symboles(texte, balise="<SYMBOLE>")** qui remplace chaque symbole par une balise.
 - Exemple : "Prix = 10€" → "Prix <SYMBOLE> 10€"
- f. Écrire une fonction **remplacer_unites(texte, balise="<UNITE>")** pour gérer les symboles d'unité : €, \$, %, km, kg, °C...
 - Exemple : "La distance est de 5 km" → "La distance est de 5 <UNITE>"
- g. Écrire une fonction **supprimer_non_alphabetiques (texte)** qui ne garde que les lettres et espaces (utile pour un nettoyage radical).
 - Exemple : "Bonjour123 !!!" → "Bonjour"
- h. Écrire une fonction **remplacer_non_alphabetiques (texte, balise="<NON_ALPHA>")** pour marquer les caractères non alphabétiques sans les supprimer.
- i. Écrire une fonction **normaliser_symboles(texte)** qui convertit certains signes en équivalents textuels :
 - + → "plus", - → "moins", = → "égal", % → "pourcent".
 - Exemple : "10% de + que l'an dernier" → "<NUM> pourcent de plus que l'an dernier"
- j. Écrire une fonction **traiter_nombres_symboles(texte, options)** qui applique sélectivement les transformations selon un dictionnaire de paramètres :

```
options = {

    "supprimer_nombres": False,

    "remplacer_nombres": True,

    "balise_nombre": "<NUM>",

    "supprimer_symboles": False,

    "remplacer_symboles": True,

    "balise_symbole": "<SYMBOLE>",

    "normaliser": True,

    "langue": "fr"}

texte_nettoye = traiter_nombres_symboles(texte, options)
```

7. Nettoyage Web et caractères spéciaux

Les textes issus du Web, des réseaux sociaux ou de bases multilingues contiennent souvent des liens URL, des adresses e-mail, des mentions, des émojis, ou encore des caractères non standard.

Ces éléments peuvent être utiles (ex. pour une analyse de sentiments) ou au contraire nuire à la comparaison des textes.

Les fonctions suivantes permettent de les identifier, supprimer ou remplacer selon les besoins.

Traitement des URLs, des adresses mails

- a. Écrire une fonction **supprimer_urls(texte)** qui supprime toutes les adresses web du texte.
 - Elle doit reconnaître les formes commençant par `http://`, `https://`, `www.` ou `ftp://`.
 - Exemple :
`"Visitez https://example.com pour plus d'infos" → "Visitez pour plus d'infos"`
- b. Écrire une fonction **remplacer_urls(texte, balise="<URL>")** qui remplace chaque lien par une balise.
 - Exemple : `"Voir le site www.openai.com" → "Voir le site <URL>"`
- c. Écrire une fonction **supprimer_emails(texte)** qui supprime toutes les adresses e-mail.
 - Exemple : `"Contactez-moi à nom@mail.com" → "Contactez-moi à "`
- d. Écrire une fonction **remplacer_emails(texte, balise="<EMAIL>")** qui remplace chaque adresse e-mail par une balise.
 - Exemple : `"Envoyez à contact@entreprise.fr" → "Envoyez à <EMAIL>"`

Traitement des mentions et des hashtags

- a. Écrire une fonction **supprimer_mentions(texte)** qui supprime toutes les mentions de type `@nom`.
- b. Écrire une fonction **remplacer_mentions(texte, balise="<MENTION>")** qui les remplace par une balise.
 - Exemple : `"Merci @user pour votre aide!" → "Merci <MENTION> pour votre aide!"`

- c. Écrire une fonction **supprimer_hashtags(texte)** pour retirer les hashtags des messages (souvent utilisés sur Twitter, Instagram, etc.).
- d. Écrire une fonction **remplacer_hashtags(texte, balise="<HASHTAG>")** pour les remplacer par une balise.

- Exemple : `"J'adore le #MachineLearning" → "J'adore le <HASHTAG>"`

Traitement des emojis

- a. Écrire une fonction **supprimer_emojis(texte)** qui supprime tous les emojis et pictogrammes.

- Exemple : `"C'était super 😊🔥" → "C'était super"`

- b. Écrire une fonction **remplacer_emojis(texte, balise="<EMOJI>")** pour les marquer sans les supprimer.

- Exemple : `"Bravo 🏆" → "Bravo <EMOJI>"`

Traitement des caractères non standard

- a. Écrire une fonction **supprimer_caracteres_speciaux (texte)** qui supprime tous les caractères invisibles ou non imprimables (tabulations, retours chariot, caractères Unicode bizarres, etc.).
- b. Écrire une fonction **remplacer_caracteres_speciaux(texte, balise="<SPEC>")** pour les remplacer par une balise.

Pipeline global de nettoyage Web et symbolique

Écrire une fonction **traiter_web_et_emojis(texte, options)** qui combine les traitements précédents selon un dictionnaire de paramètres :

```
options = {
    "supprimer_urls": True,
    "remplacer_urls": False,
    "supprimer_emails": True,
    "remplacer_mentions": True,
    "supprimer_emojis": False,
    "remplacer_emojis": True,
    "supprimer_caracteres_speciaux": True,
    "normaliser_unicode": True
}
```



```
texte_nettoye = traiter_web_et_emojis(texte, options)
```

8. Expansion et correction linguistique

Les langues française et anglaise utilisent fréquemment des formes contractées ou abrégées qui regroupent plusieurs mots en un seul. Ces formes peuvent perturber la tokenisation, la recherche de correspondances lexicales et la comparaison bilingue. L'objectif de cette section est d'écrire des fonctions capables de détecter et développer ces contractions selon la langue du texte.

Traitement et expansion des formes contractées et abréviations

- a. Écrire une fonction **expand_contractions_en(texte)** qui remplace les formes contractées anglaises par leurs équivalents complets.

- Exemple :

- "I'm" → "I am"
- "we'll" → "we will"
- "can't" → "cannot"
- "it's" → "it is"
- "they've" → "they have"

- Cette fonction peut s'appuyer sur un dictionnaire de correspondances prédéfinies, par exemple :

```
contractions_en = { "I'm": "I am", "you're": "you are",  
                    "he's": "he is", "she's": "she is", "it's": "it is", "we're": "we  
are", "they're": "they are", "I've": "I have", "we've": "we  
have", "they've": "they have", "I'll": "I will", "we'll": "we  
will", "won't": "will not", "can't": "cannot" }
```

- b. Écrire une fonction **expand_contractions_fr(texte)** qui remplace les contractions françaises avec apostrophe ou élision.

- La fonction doit être sensible à la casse (ex. "C'est" → "Ce est").

- Exemple :

- "c'est" → "ce est"
- "qu'il" → "que il"
- "j'aime" → "je aime"
- "l'homme" → "le homme"
- "s'il" → "si il"

- Un dictionnaire analogue peut être utilisé :

```
contractions_fr = { "c'": "ce ", "j'": "je ", "l'": "le ", "qu'":  
"que ", "s'": "se ", "t'": "t ", "n'": "ne ", "d'": "de " }
```

- c. Écrire une fonction **normaliser_apostrophes(texte)** qui uniformise toutes les formes d'apostrophes (', ' , `) en une seule ('), afin de faciliter la détection des contractions.
- ' (apostrophe ASCII – code U+0027)
 - ' (apostrophe typographique – code U+2019)
 - ` (guillemet ouvrant – code U+2018)
 - ` (accent grave utilisé parfois par erreur)
 - Exemple : "l'homme" → "l'homme"
- d. Écrire une fonction **expand_contractions(texte, langue="auto")** qui :
- détecte la langue automatiquement (ou selon le paramètre),
 - appelle `expand_contractions_en` ou `expand_contractions_fr` selon le cas.
 - Exemple :

```
texte_en = "We'll go if it's sunny."  
  
texte_fr = "C'est l'été, qu'il fasse beau ou non."  
  
print(expand_contractions(texte_en, "en"))  
  
# → "We will go if it is sunny."  
  
print(expand_contractions(texte_fr, "fr"))  
  
# → "Ce est le été, que il fasse beau ou non."
```

Traitement des abréviations

- a. Écrire une fonction **developper_abreviations(texte, langue="fr")** qui remplace les abréviations courantes par leurs formes complètes.
- Exemples en français :
 - "M." → "Monsieur"
 - "Dr" → "Docteur"
 - "av." → "avenue"
 - "n°" → "numéro"
 - Exemples en anglais :
 - "Mr." → "Mister"
 - "Dr." → "Doctor"
 - "St." → "Street"

○ "No." → "Number"

b. Écrire une fonction **corriger_contractions_multiples(texte)** pour traiter les cas ambigus ou combinés :

- "shouldn't've" → "should not have"
- "y'all" → "you all"
- "t'as" → "tu as"
- "p'tit" → "petit"

c. Écrire une fonction **traiter_contractions(texte, options)** pour appliquer automatiquement tous les traitements précédents selon les paramètres :

```
options = {  
  
    "langue": "auto",  
  
    "normaliser_apostrophes": True,  
  
    "expand_contractions": True,  
  
    "developper_abreviations": True  
  
}  
  
texte_nettoye = traiter_contractions(texte, options)
```

Correction typographique et normalisation lexicale

- a.** Écrire une fonction **corriger_fautes_typographiques(texte)** qui détecte et corrige les répétitions de lettres ou les fautes simples.
- b.** Écrire une fonction **uniformiser_variantes(texte)** qui harmonise les formes lexicales proches (“covid19”, “covid-19”, “COVID” → “covid”).

9. Normalisation des espaces et de la mise en forme

- a.** Écrire une fonction **supprimer_espaces_multiples(texte)** qui remplace toute suite d’espaces (ou tabulations) par un seul espace.
- Exemple :
"Bonjour à tous" → "Bonjour à tous"
 - Suppression des espaces en début et fin de ligne
- b.** Écrire une fonction **supprimer_espaces_bords(texte)** qui supprime les espaces inutiles au début et à la fin de la chaîne (ou de chaque ligne).

- Exemple :
" Bonjour à tous " → "Bonjour à tous"

- c. Écrire une fonction **normaliser_retours_ligne(texte)** qui :
- remplace les différentes formes de sauts de ligne (`\r`, `\r\n`, `\n`) par une seule (`\n`),
 - Exemple :
"Bonjour\r\nComment ça va ?" → "Bonjour\nComment ça va ?"
- d. Écrire une fonction **supprimer_lignes_vides(texte)** qui supprime les lignes vides ou les successions de plusieurs retours à la ligne.
- Exemple :

Bonjour ⇒ "Bonjour\nComment ça va ?" Comment ça va ?

- e. Écrire une fonction **remplacer_tabulations(texte, nb_espaces=1)** qui remplace chaque tabulation (`\t`) par un nombre défini d'espaces.
- Exemple :
"Nom\tPrénom\tÂge" → "Nom Prénom Âge"
- f. Écrire une fonction **supprimer_espaces_avant_ponctuation (texte)** qui supprime les espaces mal placés avant les signes de ponctuation (`.`, `,`, `;`, `:`, `?`, `!`).
- Exemple : "Bonjour , comment ça va ?" → "Bonjour, comment ça va?"
- g. Écrire une fonction **ajouter_espace_apres_ponctuation(texte)** qui s'assure qu'il y a exactement un espace après chaque ponctuation.
- Exemple : "Bonjour,comment ça va?" → "Bonjour, comment ça va?"
- h. Écrire une fonction **nettoyer_espaces(texte)** qui combine plusieurs fonctions précédentes :
- suppression des espaces multiples,
 - suppression des espaces avant ponctuation,
 - ajout d'un espace après ponctuation,
 - suppression des espaces de début et de fin.
- i. Écrire une fonction **normaliser_mise_en_forme(texte, options)** qui applique les traitements selon des paramètres choisis :

<pre>options = { "remplacer_tabulations": True, "supprimer_espaces_multiples": True,</pre>
--

```

"supprimer_lignes_vides": True,

"ajouter_espace_apres_ponctuation": True,

"supprimer_espaces_avant_ponctuation": True,

"normaliser_retours_ligne": True

}

texte_nettoye = normaliser_mise_en_forme(texte, options)

```

10. Vérifications de cohérence post-nettoyage

- a. Écrire une fonction **detecter_texte_vide(texte)** qui vérifie si un document est devenu vide après nettoyage (ou ne contient plus que des espaces).
 - Exemple :
" " → détecté comme vide.
 - Elle renverra un booléen (`True` si le texte est vide) ou un message d'alerte.
- b. Écrire une fonction **comparer_longueurs_texte(texte_avant, texte_apres, seuil=0.3)** qui compare la longueur du texte avant et après nettoyage, afin de détecter une perte excessive d'information.
 - Exemple :
Si `len(texte_apres) < 30% de len(texte_avant)`, signaler une alerte.
 - Cette fonction permet de s'assurer que le nettoyage n'a pas supprimé trop de contenu utile.
- c. Écrire une fonction **signaler_textes_vides(base)** qui parcourt l'ensemble du corpus et liste les fichiers devenus vides après nettoyage, pour permettre une inspection manuelle.

11. Production et sauvegarde du texte nettoyé

Après avoir appliqué toutes les étapes de prétraitement et vérifié la cohérence du corpus, il est recommandé de produire une version finale du texte nettoyé et de l'enregistrer dans un répertoire dédié. Cela facilite l'analyse, la tokenisation et l'expérimentation ultérieure.

- a. Écrire une fonction **nettoyer_document** qui combine toutes les étapes précédentes et retourne un texte propre et homogène, prêt à être tokenisé.
 - Exemple de signature et d'usage :

```

texte_nettoye = nettoyer_document(fichier, options={"ponctuation": {...},
"nombres": {...},
"symboles": {...},
"web": {...},
"contractions": {...},
"mise_en_forme": {...}}

```

- b. Ecrire une fonction **sauvegarder_texte_propre** qui enregistre le texte nettoyé dans un répertoire dédié au corpus prétraité, en conservant le nom du fichier original.
 - Exemple :
`sauvegarder_texte_propre(fichier, texte_nettoye, dossier_sortie = "corpus_nettoye")`
- c. Appliquer **nettoyer_document** à tous les fichiers du mini-corpus.
 - a. Vérifier que les textes ne sont pas vides et que les transformations ont été correctement appliquées.
 - b. Sauvegarder tous les textes nettoyés avec **sauvegarder_texte_propre** dans un nouveau dossier **corpus_nettoye**.
 - c. Vérifier que la structure des fichiers est cohérente et prête pour l'étape de tokenisation ou d'indexation.

Sources utiles :

Pour approfondir vos connaissances et disposer des outils nécessaires à cette séance, vous pouvez consulter les ressources suivantes :

<https://docs.python.org/3/library/unicodedata.html>

<https://docs.python.org/3/library/string.html>

<https://pypi.org/project/num2words/>

<https://docs.python.org/fr/3/howto/regex.html>

<https://python.doctor/page-expressions-regulieres-regular-python>

https://www.w3schools.com/python/python_regex.asp